Implementation Notes

1. Java8 was assumed.
2. I wanted to include a command line interface.
3. The maven project is split in to 2 modules. One module being the core of the implementation, and the other being the command line interface. This was done to separate concerns; it felt wrong to include a command line library dependency (info.picocli) in the core module.
4. I chose to allow the word counter to read from a URI. This allows it to read files directly from web addresses as well as local file system.
5. UTF-8 character set is used. I would probably modify it to support any character set, given more time.
6. In a real world setting I would probably include logging.
7. In a real world setting the POMs would be setup to allow deployment to a repository.
8. For scalability I used a scanner to read the files, as this streams and keeps a low memory footprint even for large files. Reading the whole file in to memory would not scale well.
9. For scalability I also included a multi-threaded version of the word counter. For small files the single threaded version runs quicker as it doesn't have the overhead of handling the thread pool. But for larger files I see a speed improvement with the multi-threaded version.
10. A regex was used to split lines of text into words. I'm happy that the regex does what I intended, but I might like to see if there's a way to simplify it.

<u>What defines a word?</u>

A word is any number of characters that appear between word delimiters.

I used the following characters as word delimiters:

- space
- exclamation
- question mark
- colon
- semi-colon
- comma
- period

Note: Commas and periods are a special case. They shouldn't be considered as delimiters if they appear within a formatted number. E.g. "3,500.75" should be treated as one word, and not split in to 3 words ["3", "500", "75"]

However, a comma or a period appearing at the beginning or end of a number should be treated as a delimiter. E.g. ",3.500.75." should yield one word of "3.500.75" and not a word of ",3.500.75."

More refinements could be made. For example, the removal of speech marks and other characters which would incorrectly be included in words. There might be linguistic libraries available that could help.

Installation Instructions

1. Clone the repository https://github.com/smitchell100/wordcounter.git into a directory of your choice.
2. Run the following command from that directory:

```
mvn package
```

Programming API

If the real world, the module would be deployed to a maven repository, in which case a dependency could be added to your project as follows:

```
<dependency>
  <groupId>com.synalogik</groupId>
  <artifactId>wordcount-core</artifactId>
  <version>1.0-SNAPSHOT</version>
</dependency>
```

Alternatively, include the generated jar file in your project classpath (found at *wordcountcore/target/wordcount-core-1.0-SNAPSHOT.jar*).

Two flavours of word counter are provided; single-threaded (better suited for small files) and multi-threaded (better suited for large files).

Single-threaded example

```
import com.synalogik.wordcountcore.WordCounter;
import com.synalogik.wordcountcore.SingleThreadedWordCounter;
import com.synalogik.wordcountcore.model.WordCountMetrics;
import java.net.URI;
.
.
.
URI pathToTextSource = new URI("https://janelwashere.com/files/bible_daily.txt");
WordCounter wordCounter = new SingleThreadedWordCounter();
WordCountMetrics metrics = wordCounter.analyseText(pathToTextSource);
```

Multi-threaded example

```
import com.synalogik.wordcountcore.WordCounter;
import com.synalogik.wordcountcore.MultiThreadedWordCounter;
import com.synalogik.wordcountcore.model.WordCountMetrics;
import java.net.URI;
.
.
.
int numberOfWorkerThreads = 3;
int workerQueueSize = 100;
URI pathToTextSource = new URI("https://janelwashere.com/files/bible_daily.txt");
WordCounter wordCounter = new MultiThreadedWordCounter(numberOfWorkerThreads, workerQueueSize);
WordCountMetrics metrics = wordCounter.analyseText(pathToTextSource);
```

Both examples return a WordCountMetrics object, offering the following methods from which word count metrics can be retrieved:

Method	Description
boolean **hasData**()	Was any data captured?
Double **getAverageWordLength**()	The average word length across the whole text
int **getTotalNumberOfCharactersInWords**()	Total number of characters in all words scanned
int **getTotalWordCount**()	Total number of words in whole text
OptionalInt **getHighestFrequencyOfAWordLength**()	Which word length occurred most in whole text
List<Integer> **getWordLengthsHavingFrequencyOf**(int frequency)	Which word lengths occurred a given number of times
Map<Integer, Integer> **getMapFrequencyOfWordsGroupedByWordLength**()	Get the entire mapping of word lengths to frequency

Because the WordCounter interface takes a URI path to the text file, local file system files can also be used in addition to the web url example given. E.g.

```
URI pathToTextSource = new URI("file:///path/to/file/filename.txt");
```

Rendering the results

The results contained in a WordCountMetrics object can be printed to System.out by using a MetricsRenderer:

```
import java.io.StringWriter;
import com.synalogik.wordcountcore.rendering.MetricsRenderer;
import com.synalogik.wordcountcore.rendering.WriterMetricsRenderer;
.
.
.
StringWriter sw = new StringWriter();
MetricsRenderer renderer = new WriterMetricsRenderer(sw);
renderer.renderMetrics(metrics);
System.out.println(sw.toString());
```

Command Line Interface

A command line interface is also provided, offering single threaded and multi-threaded operation.

To run it, first ensure that you are in the directory where the zip file was expanded (and mvn command executed).

Run:

```
java -jar wordcountcli/target/wordcount-cli-1.0-SNAPSHOT-jar-with-dependencies.jar -h
```

to view the command line help, which is as follows:

```
Usage: wordMetrics [-hV] [-q=<workerQueueSize>] [-t=<numberOfWorkerThreads>] <pathToTextSource>

Displays word count metrics for a given text source.

    <pathToTextSource>   The path to the text source.

 -h, --help            Show this help message and exit.

 -q, --queueSize=<workerQueueSize>

                  (Only applies when number of worker threads > 1).

                  The worker queue size used when streaming the given file (default is 100).

 -t, --threads=<numberOfWorkerThreads>

                  The number of worker threads used when analysing files (default is 1).

 -V, --version         Print version information and exit.
```

Results are written to StdOut.

Example Usage against a web hosted text file:

```
java -jar wordcountcli/target/wordcount-cli-1.0-SNAPSHOT-jar-with-dependencies.jar -t 5 https://janelwashere.com/files/bible_daily.txt
```

This will read the text from the web address given, running with 5 threads.

Example Usage against a local text file:

```
java -jar wordcountcli/target/wordcount-cli-1.0-SNAPSHOT-jar-with-dependencies.jar file:///path/to/file/filename.txt
```

This will read the text from the local file system, using a single thread.